

Relatório do Trabalho Final

Guilherme Melos Vieira

1. Estrutura de Arquivos

O projeto é dividido nos seguintes arquivos:

- `main.py` — ponto de entrada do programa. Lê o arquivo de configuração e inicia o nó.
- `node.py` — núcleo da aplicação. Contém a classe `Node`, com toda a lógica de inicialização, recebimento e tratamento de pacotes, envio para o sucessor, controle do token e o loop de comandos do usuário.
- `packets.py` — define os tipos de pacotes (`DISCOVER`, `HELLO`, `TOKEN`, `DADOS`) e as funções de construção (`build_*`) e interpretação (`parse_packet`) das mensagens trocadas na rede.
- `topology.py` — representa a topologia do anel (dicionário de apelido para IP) e calcula o sucessor de cada máquina em ordem alfabética.
- `crc.py` — cálculo e verificação do CRC32 das mensagens.
- `fault.py` — módulo de inserção de falhas, que corrompe mensagens de acordo com uma probabilidade configurada.
- `listener.py` — thread responsável por receber pacotes UDP continuamente e repassá-los para o tratamento no nó.
- `token_monitor.py` — thread responsável por detectar perda do token (timeout) na máquina controladora.
- `ui.py` — funções de entrada e saída: leitura de comandos, exibição de mensagens, gravação de logs e exibição de topologia/fila.
- `init.txt` — arquivo de configuração lido na inicialização (apelido, tempos, probabilidade de erro).

2. Arquivo de Configuração

O arquivo de inicialização `init.txt` contém cinco linhas, lidas em `create_node`, em `node.py`:

- Linha 1: apelido da máquina (ex.: A, B, C...)
- Linha 2: tempo de token e de dados, usado como atraso de envio (`token_data_time`)
- Linha 3: probabilidade de inserir erro nas mensagens, em percentual (`error_probability`)
- Linha 4: timeout do token, usado pela máquina controladora para detectar perda (`token_timeout`)
- Linha 5: tempo mínimo entre tokens, usado para detectar token duplicado (`min_token_interval`)

Exemplo de conteúdo, referente à máquina A:

```
A
2
20
2,5
2
```

Os valores numéricos usam vírgula como separador decimal, por isso a função `create_node` troca a vírgula por ponto antes de converter para `float`.

3. Estruturas de Dados e Classes

3.1 Classe Node

A classe Node (node.py) é uma dataclass que representa o nó local. Os atributos definidos na criação (nickname, token_data_time, error_probability, token_timeout, min_token_interval) vêm do arquivo de configuração. No método start, outros atributos de estado são inicializados:

- ip — endereço IP local, obtido em `_get_local_ip` abrindo uma conexão UDP de teste para descobrir qual interface seria usada.
- topology — instância de Topology, inicialmente contendo apenas o próprio nó.
- queue — fila de mensagens a enviar, com capacidade máxima de 10 itens (`queue.Queue(maxsize=10)`).
- socket — socket UDP, criado com `SO_REUSEADDR` e `SO_BROADCAST` habilitados, vinculado à porta 6000.
- running — flag que controla o funcionamento das threads e do loop principal.
- waiting_data — indica que uma mensagem foi enviada e a máquina está esperando ela voltar pelo anel.
- remove_token — flag setada pelo comando do usuário para retirar o próximo token recebido.
- controls_token — indica se esta máquina é a controladora do token (responsável por detectar perda ou duplicação).
- last_token_time — instante da última passagem do token pela controladora.

3.2 Classe Topology

Definida em topology.py, é uma dataclass que guarda um dicionário {apelido: ip} das máquinas conhecidas. O método `get_successor` ordena os apelidos alfabeticamente e retorna o próximo da lista.

3.3 Fila de mensagens

Cada item da fila é um dicionário com os campos `destination` (apelido do destino ou BROADCAST), `message` (texto da mensagem) e `tries` (número de tentativas de envio já realizadas). A fila é populada pelo comando "mensagem <destino> <texto>" digitado pelo usuário (tratado em `_add_message`) e consumida em `_send_queued_message` quando o token chega e há mensagens pendentes.

4. Threads e Mecanismos de Sincronização

A aplicação utiliza três linhas de execução concorrentes além da thread principal:

- Thread de escuta (listener.py, função `listen`) — fica em loop chamando `socket.recvfrom`, decodificando o pacote recebido e chamando `node._handle_packet`. Usa timeout de 0,5s no socket para poder verificar periodicamente a flag `running` e encerrar de forma limpa.
- Thread de monitoramento do token (token_monitor.py, função `monitor_token`) — ativa apenas na máquina controladora (`controls_token`). A cada 0,5s verifica se o tempo desde a última passagem do token (`last_token_time`) excedeu `token_timeout`; se sim, gera e envia um novo token, tratando o caso de token perdido.

- Thread principal (`node._command_loop`) — lê comandos do usuário (mensagem, token adicionar, token remover, topologia, fila, logs, ajuda, sair) e atualiza o estado do nó.
- Threads de envio com atraso (`_send_to_successor_after_delay`) — disparadas a cada envio de pacote que não seja token controlado pela própria máquina, para aplicar o tempo configurado (`token_data_time`) sem bloquear quem chamou o envio.

Como tem vários pontos do código (`listener`, monitor de token, comandos do usuário) que podem alterar o estado do nó ao mesmo tempo, a aplicação usa seguras para concorrência: como `queue.Queue`, que já é `thread-safe`, para a fila de mensagens, e um `Lock` (`LOG_LOCK`, em `ui.py`) para evitar que duas threads escrevam ao mesmo tempo e corrompam o arquivo.

5. Inicialização do Anel (DISCOVER / HELLO)

Ao iniciar (método `start`), o nó descobre seu próprio IP, cria o socket, inicia as threads de escuta e monitoramento, e envia três vezes (`DISCOVERY_ATTEMPTS`) um pacote DISCOVER em broadcast, com um intervalo de 1 segundo entre cada envio, para aumentar a chance de outras máquinas responderem.

Quando um nó recebe um DISCOVER de outra máquina (`_handle_discover`), ele cadastra o remetente na sua topologia e responde com HELLO, tanto diretamente para o IP de origem quanto por broadcast. Quando um nó recebe um HELLO (tratado em `_add_machine` via `_handle_packet`), ele adiciona ou atualiza o IP do remetente na topologia.

A cada atualização da topologia, o método `_analyze_token_controller` recalcula quem é a máquina controladora do token: por definição, é sempre a primeira em ordem alfabética entre as conhecidas.

Após as tentativas de DISCOVER, `_create_first_token_if_needed` verifica se esta máquina é a controladora e, se for o caso, gera o primeiro token e o envia ao sucessor.

6. Circulação do Token

Quando um TOKEN é recebido (`_handle_token`), o nó primeiro verifica se foi solicitada a remoção do token pelo comando do usuário (`remove_token`); se sim, o token é descartado.

Se a máquina é a controladora do token, é feita a verificação de token duplicado e a atualização de `last_token_time` que serve para a detecção de perda de token.

Se a máquina está esperando o retorno de uma mensagem de dados enviada anteriormente (`waiting_data`), o token é apenas repassado ao sucessor.

Se a fila de mensagens está vazia, o token também é apenas repassado. Caso contrário, a primeira mensagem da fila é enviada (`_send_queued_message`), e o token só volta a circular depois que os dados deram a volta completa e o nó não tiver mais mensagens para enviar.

O comando "token adicionar" permite gerar e inserir manualmente um novo token na rede, e o comando "token remover" agenda a retirada do próximo token recebido.

7. Envio e Recebimento de Dados

Ao montar uma mensagem de dados para envio (`_send_queued_message`), é calculado o CRC32 da mensagem original (`calculate_crc`, em `crc.py`, que usa `zlib.crc32`). Em seguida, se o destino não for BROADCAST e for a primeira tentativa de envio, o módulo de inserção de falhas (`maybe_corrupt`, em `fault.py`) pode corromper a mensagem, de acordo com a

probabilidade configurada (`error_probability`). O pacote é então montado com o campo de controle inicial "maquinainexistente", conforme exigido na definição.

Ao receber um pacote de dados (`_handle_data`), o nó verifica três situações: se o apelido de origem é o seu próprio (a mensagem completou o anel e voltou — tratado em `_handle_returned_data`), se o apelido de destino é o seu próprio (a mensagem é para ele — tratado em `_handle_my_data`) ou se o destino é BROADCAST (mensagem para todos). Em qualquer outro caso, o pacote é simplesmente repassado ao sucessor (`_forward_data`).

No destino (`_handle_my_data`), o CRC é recalculado sobre a mensagem recebida e comparado ao CRC enviado (`is_valid_crc`, em `crc.py`). Se for válido, o campo de controle é alterado para ACK; se inválido, para NAK. O pacote, já com o controle atualizado, é repassado ao sucessor para retornar à origem.

Na origem (`_handle_returned_data`), o tratamento depende do campo de controle recebido:

- ACK: a mensagem é retirada da fila (`queue.get()`) e o token é liberado para o sucessor.
- NAK, primeira ocorrência (`tries < 2`): a mensagem não é retirada da fila, sendo retransmitida na próxima passagem do token e sem possibilidade de inserção de falhas, e o token é liberado.
- NAK, segunda ocorrência: a mensagem é descartada da fila (não é pra acontecer nunca, só em caso de erro de implementação do CRC32), e o token é liberado.
- maquinainexistente (destino não encontrado na rede) ou broadcast que completou a volta: a mensagem é retirada da fila e o token é liberado, com aviso na tela.

Em todos os casos, a flag `waiting_data` é desativada e um novo TOKEN é enviado ao sucessor somente depois desse tratamento, atendendo à regra de que o token só volta a circular após o retorno completo dos dados.

8. Módulo de Inserção de Falhas e Controle de Erro (CRC32)

O arquivo `fault.py` implementa a inserção de falhas. A função `should_corrupt` faz a aleatorização da inserção de falhas com a chance definida no `init.txt`. A função `corrupt_message` corrompe a mensagem de forma simples, acrescentando o sufixo "#ERRO" ao final do texto, o que faz o CRC recalculado no destino não corresponder ao CRC enviado. A função `maybe_corrupt` junta as duas etapas e é a função efetivamente chamada pelo nó antes de montar o pacote de dados.

O cálculo de CRC (`crc.py`) usa a função `crc32` da biblioteca padrão `zlib` sobre os bytes da mensagem (codificados em UTF-8), aplicando uma máscara de 32 bits. A verificação (`is_valid_crc`) recalcula o CRC da mensagem recebida e compara com o valor informado no pacote.

9. Envio Unicast e Broadcast

O envio de pacotes ao sucessor sempre ocorre por unicast UDP, direcionado ao IP do próximo nó do anel (`_send_to_successor_now`, usando `topology.get_successor`). Já o envio em broadcast (`_broadcast`) é usado apenas durante a descoberta da rede (DISCOVER e HELLO), sendo enviado ao endereço de broadcast da subrede, calculado a partir do IP local e da máscara de rede (`_broadcast_addresses` e `_get_netmask`, com fallback para 255.255.255.255 caso a máscara não seja obtida).

Mensagens de dados endereçadas ao apelido especial BROADCAST trafegam pelo anel da mesma forma que uma mensagem unicast (de nó a nó, via `_forward_data`), mas são exibidas para todas as máquinas pelas quais passam e mantêm o campo de controle

sempre em "maquinainexistente" durante o percurso, conforme exigido na definição do trabalho.

10. Entrada de Novas Máquinas no Anel

Como a topologia (`self.topology.machines`) é apenas um dicionário em memória, atualizado a cada DISCOVER ou HELLO recebido, uma nova máquina pode entrar no anel em qualquer momento sem necessidade de reiniciar as demais. Ao iniciar, a nova máquina envia seu próprio DISCOVER; as máquinas já ativas respondem com HELLO e atualizam sua topologia (recalculando também, se necessário, qual máquina passa a controlar o token, já que a ordenação alfabética pode mudar). A partir daí, o cálculo do sucessor (`Topology.get_successor`) já considera a nova máquina automaticamente.

11. Interface com o Usuário e Logs

O módulo `ui.py` centraliza toda a entrada e saída de texto. Os comandos disponíveis no loop principal são:

- mensagem <destino|BROADCAST> <texto> — adiciona uma mensagem à fila de envio
- token adicionar — insere manualmente um novo token no anel
- token remover — agenda a retirada do próximo token recebido
- topologia — exibe a ordem do anel e o IP de cada máquina
- fila — exibe as mensagens pendentes de envio e o número de tentativas
- logs — exibe as últimas linhas do log da máquina
- ajuda — lista os comandos disponíveis
- sair — finaliza a aplicação

Cada evento relevante (envio e recebimento de pacotes, mudanças de topologia, controle do token, ACK/NAK, retransmissões, mensagens corrompidas, comandos executados) é registrado em um arquivo de log, dentro da pasta `logs/`, com data e hora, por meio da função `ui.log`, protegida por um Lock para evitar concorrência na escrita. Isso permite acompanhar detalhadamente o que a máquina está fazendo durante a execução, inclusive eventos de token perdido ou duplicado. Eventos mais importantes como recebimento de mensagens são printados direto na tela